

论文格式审查系统 · 完整框架

一、论文拆解

论文 docx 按以下7个单元拆解：

编号	单元名称
1	封面
2	扉页
3	摘要
4	章节标题（一级标题、二级标题.....）
5	图/表/公式编号
6	正文
7	参考文献列表

二、代码 vs Agent 分界原则

核心原则：能用纯代码判定的，绝不交给大模型。

7个单元中，6个纯代码，只有参考文献的著录内容格式需要Agent介入：

编号	单元名称	使用方式	判定理由
1	封面	纯代码	固定位置、固定字体字号，直接判等
2	扉页	纯代码	同上
3	摘要	纯代码	标题格式固定，字数用代码统计，关键词格式用正则匹配
4	章节标题	纯代码	层级编号用正则提取，字号字体用docx属性判等，规则是死数据
5	图/表/公式编号	纯代码	编号连续性做数学判断，格式用正则匹配，不需要理解语义
6	正文	纯代码	全文属性比对，不需要理解文字含义
7	参考文献列表	部分代码 + 部分Agent	前半段：列表提取、编号顺序检查 → 纯代码 后半段：GB/T 7714-2025 著录格式合规判断 → 唯一必须用Agent的地方

三、分工方案（6人）

核心原则：每个人工作量均衡，并且基本上每个人工作是并发的

负责部分	人数	职责
文档拆解(parser)	1	用python-docx读取docx，按7个单元拆解并输出统一结构化数据（JSON），供所有审查使用
审查1(checker1)	1	负责单元1（封面）、2（扉页）、3（摘要）的纯代码审查：字体、字号、对齐、字数限制、关键词格式
审查2(checker2)	1	负责单元4（章节标题）、5（图/表/公式编号）、6（正文）的纯代码审查：标题层级、编号连续性、全文一致性
参考文献审查-代码侧(ref-code)	1	①编号顺序检查（纯代码） ②研究GB/T 7714-2025标准，将著录规则抽象成JSON结构，供Agent侧使用
参考文献审查-Agent侧(ref-agent)	1	调用已部署的本地大模型 API + 提示词设计与调优 + 将JSON规则嵌入提示词 + 著录合规判断 + 30条测试集标注与迭代

负责部分	人数	职责
规则制定 + 报告汇总(rules)	1	前期： 将学院论文规范要求，提炼分块整合，变成规则清单，将对应部分给到相应的审查者，作为审查者编码依据 后期： 收集所有报错，生成最终问题定位报告 + 构建20份错误注入测试集 + 计算查全率/误报率

四、整体流程与各个部分衔接（所有人写代码之前必须完成的动作）

原则：**保证并行，尽可能减少前后依赖。**

谁做	产出什么	给谁
规则制定者	规则清单（分7个单元的中文条目）	分给审查1、审查2、参考文献审查（代码侧）
审查1	报错JSON模板（预期结构样例）	给汇总者
审查2	报错JSON模板（预期结构样例）	给汇总者
参考文献审查（代码侧）	报错JSON模板 + 著录规则JSON模板	报错模板给汇总者；著录规则JSON给Agent侧
参考文献审查（Agent侧）	报错JSON模板（预期结构样例）	给汇总者
文档拆解	统一结构化数据JSON模板（7个单元的输出格式样例）	给所有审查者

注1：完成上述准备后，所有人并行开工写代码，此时每个人的输入/输出格式已经全部约定好，互不阻塞，独立开发。

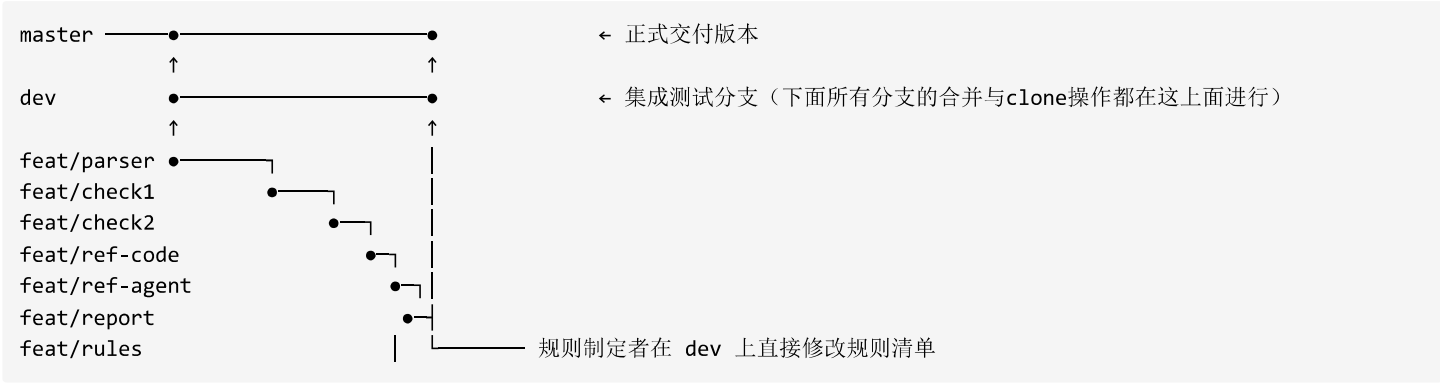
注2：规则制定者把规则清单分发给审查者时，按单元切分：

- 封面/扉页/摘要规则 → 给审查1
- 章节/编号/正文规则 → 给审查2
- 参考文献相关规则（编号格式 + GB/T 7714-2025字段说明） → 给参考文献审查（代码侧）

注3：规则制定者产出规则清单是唯一一个前置动作，一定要最先完成。

五、Git 分支策略

分支结构（三层）



六、仓库目录结构

论文格式审查系统/	
├─ run.py	
├─ src/	
│ ├─ parser/	# 【唯一入口】对外发布的主运行文件
│ │ ├─ docx_parser.py	# 运行方式: python run.py 论文.docx -c config.toml
│ │ └─ unit_defs.py	# 职责: 读取配置 → 自动建目录 (output/、result/)
│ └─ checkers/	# → 调用 parser → 调用 checkers → 调用 report
│ ├─ cover_checker.py	
│ ├─ abstract_checker.py	# 【源代码目录】对外发布
│ ├─ heading_checker.py	# 【文档拆解 x1人】
│ ├─ figure_checker.py	# 主拆解器: 按行文顺序遍历 docx, 识别各单元并归类
│ ├─ body_checker.py	# 7 个单元的识别规则 (样式名对照、正则匹配规则等)
│ └─ references/	# 【审查模块】
│ ├─ code_checker.py	# 【审查1】封面+扉页格式检查 (字体、字号、对齐方式)
│ ├─ agent_checker.py	# 【审查1】摘要格式检查 (字数统计、关键词格式)
│ └─ rules/	# 【审查2】章节标题检查 (层级编号顺序、字号字体)
│ └─ gbt7714_2025.json	# 【审查2】图/表/公式编号检查 (连续性、编号格式)
└─ report/	# 【审查2】正文格式检查 (全文样式一致性)
└─ reporter.py	# 【参考文献审查】
	# 【代码侧】参考文献编号顺序检查 (递增、无跳号)
├─ output/	# 【Agent侧】GB/T 7714-2025 著录合规判定 (调用本地LLM)
│ ├─ parser_json/	# 著录规则定义
│ └─ checker_json/	# 从标准中提取的著录规则 JSON (供 Agent 侧加载)
	# 【报告汇总 x1人】
├─ result/	# 合并 N 份审查报错 JSON → 生成最终问题定位报告
├─ config.toml	# 【dev 全量提交】中间产物 + JSON 模板存放目录
├─ tests/	# 拆解后的结构化论文数据
│ ├─ fixtures/	# 各审查模块产出的报错
│ │ ├─ normal.docx	# reporter 直接读取 checker_json 进行汇总
│ │ └─ injected/	
│ │ ├─ error_001.docx	# 【dev 全量提交】最终报告存放目录 (格式待定)
│ │ ├─ error_002.docx	
│ │ └─ ...	
└─ run_tests.py	# 【对外发布】配置文件
	# 【仅 dev】测试目录
	# 测试用 docx 文档
	# 合格论文模板 (格式完全正确的对照标准)
	# 错误注入文档 (共 20 份, 覆盖各类错误类型)
├─ docs/	# 测试运行脚本
│ ├─ 架构设计/	# 遍历 fixtures 下所有 docx → 跑完整审查流程
│ │ └─ 整体架构.md	# → normal 应 0 报错, injected 应报出预期错误
│ └─ 规范规则/	# → 输出查全率 / 误报率统计
│ ├─ 学院论文规范/	
│ └─ gbt7714_2025.md	# 【仅 dev】团队内部文档目录 (不对外发布)
	# 架构设计说明
	# (就是这个文件) 拆解方式、分工方案、处理流程
	# 论文规范规则原文
	# 学院发布的论文格式要求原文
	# GB/T 7714-2025 摘录说明
├─ .gitignore	# 排除规则: __pycache__/ .venv/ .env/ 等运行环境产物
├─ pyproject.toml	# 【对外发布】开发工具统一配置 (ruff 规则、pytest 参数等)
├─ requirements.txt	# 【对外发布】Python 依赖清单 (python-docx、httpx 等)
└─ README.md	# 【对外发布】给用户看的说明 (功能简介、运行命令、模块负责人一览)

各分支职责

分支	用途	说明
master	正式发布	只包含交付给用户的文件（run.py + src/ + config.toml + requirements.txt + pyproject.toml + README.md）
dev	集成测试	包含全部文件（src/ + tests/ + docs/ + 配置等），各 feat 分支合并到这里进行全量测试和整体联调
feat/*	个人开发	每人一个分支，互不阻塞。 命名规则：feat/parser、feat/check1、feat/check2、feat/ref-code、feat/ref-agent、feat/repor
output/ 和 result/	存储产物	开发阶段存放各模块的 JSON 模板和中间产物，随 dev 分支全量提交，以便团队成员共享接口数据。

七、git协作流程

首次导入与创建分支

```
# 1.打开一个空文件夹,右键选择Open Git Bash here

# 2.克隆仓库地址
git clone https://github.com/yuhy66/AI-Paper-Format-Review.git

# 3.关掉命令终端，找到克隆下来的文件夹，打开，然后右键重新打开git bash

# 4.由于这时候默认打开的是master,所以切换到dev
git switch dev

# 5.基于本地的 dev，创建成员自己的功能分支 feat/xxx
git switch -c feat/parser
# 6.之后在这条分支上面进行编程，上传到暂存区，仓库等所有操作
```

合并与推送

```
# 开发到一定程度后想合并到dev，需要先把本地dev更新到最新
# 1. 切换到 dev 并拉取团队的最新代码
git checkout dev
git pull origin dev

# 2. 切回你自己的 feat 分支
git checkout feat/parser

# 3. 把最新的 dev 代码合并到你的 feat 分支中（这一步可能会产生冲突）
git merge dev

# --- 如果第3步提示有冲突（CONFLICT） ---
# 在你的代码编辑器里手动解决冲突文件。
# 解决完之后，执行以下命令标记解决完成：
git add <已解决冲突的文件>
git commit -m "解决***的冲突"
# -----

# 4. 冲突解决完毕，切回 dev，把你的 feat 合并进来
git checkout dev
git merge feat/parser

# 5. 核心补充将合并好之后的最新 dev 推送到远程仓库
git push origin dev
```

正式版本发布

```
# 在dev上面跑通了之后，往master合并时，只需要合并需要对外发布的文件

# 1.切换到master
git switch master

# 2.直接把 dev 分支里与你交付相关的核心文件，覆盖到当前 master 工作区
git restore --source=dev -- run.py src/ config.toml requirements.txt pyproject.toml README.md

# 3.接下来正常上传操作
git add .
git commit -m "日志（谁做了什么）"
git push origin master

# 4.额外操作，附加版本号与GitHub发布（通常由组织者操作）
git tag -a <tag-name> -m "<message>" <commit-id>
# 例： git tag -a v1.0 -m "论文格式审查1.0版本" 9d7de85
# 之后在GitHub上面release即可
```

八、全员通用约定

约定	说明
语言	代码注释用中文，变量/函数命名用英文
命名风格	函数/变量 snake_case，类 PascalCase
类型注解	每个函数必须有输入输出类型注解（Type Hints）
模块接口	每个模块对外暴露的函数必须有明确的输入输出类型注解
提交方式	每次提交必须要有日志说明做了什么事